

API Pentesting

Introduction

API penetration testing is a critical security assessment process that evaluates application program interfaces to ensure they meet security, functionality, and performance standards. While web testing focuses on user interface elements and browser-based vulnerabilities, API testing directly analyzes endpoints and request/response structures.

Core Concepts

- **API Architecture and Documentation**

- API architecture serves as a blueprint for designing web services, outlining how components connect to fulfill specific tasks.
- Documentation is essential for pen-testing, providing a detailed guide to endpoints and parameters to identify risks effectively.

- **REST vs SOAP Protocols**

- REST (Representational State Transfer) is a flexible architectural style using standard HTTP methods like GET and POST.
- SOAP (Simple Object Access Protocol) is a protocol that uses XML for structured messaging and defines its own security standards.
- The pentesting process includes stages of Planning, Discovery, Testing, Reporting, and Remediation.

- **Vulnerability Classes**

- BOLA (Broken Object Level Authorization) occurs when an API fails to validate user access to specific data objects.
- Mass Assignment vulnerabilities allow attackers to modify sensitive internal variables through API input parameters.

Impact and Vulnerabilities

- **Authorization and Access Risks**

- Improper implementation of BOLA or BFLA (Broken Function Level Authorization) can lead to full account takeover or data breaches.

- **Authentication and Information Disclosure**

- Broken authentication mechanisms allow attackers to compromise tokens and bypass security checks.
- Information disclosure through improper error handling can leak API keys or database versions.

Case Study: Mass Assignment in a Social Media API

Exploiting Auto-Binding Logic

A social media application utilized an API that automatically bound incoming JSON parameters to user profile objects without filtering sensitive fields.

The Vulnerability

Testing revealed that by adding an additional parameter, such as `"is_admin": true`, to a standard profile update request, the API would incorrectly update the user's administrative status.

The Attack (Request Manipulation)

Using Postman, researchers intercepted a `PUT` request to the `/api/v1/profile` endpoint. By injecting the unauthorized parameter into the JSON body, they successfully escalated their privileges to an administrator account.

Result

The privilege escalation allowed the researchers to access global user data, highlighting the critical need for explicit input validation and object mapping.

Practical (Lab Setup: Postman & VAmPI)

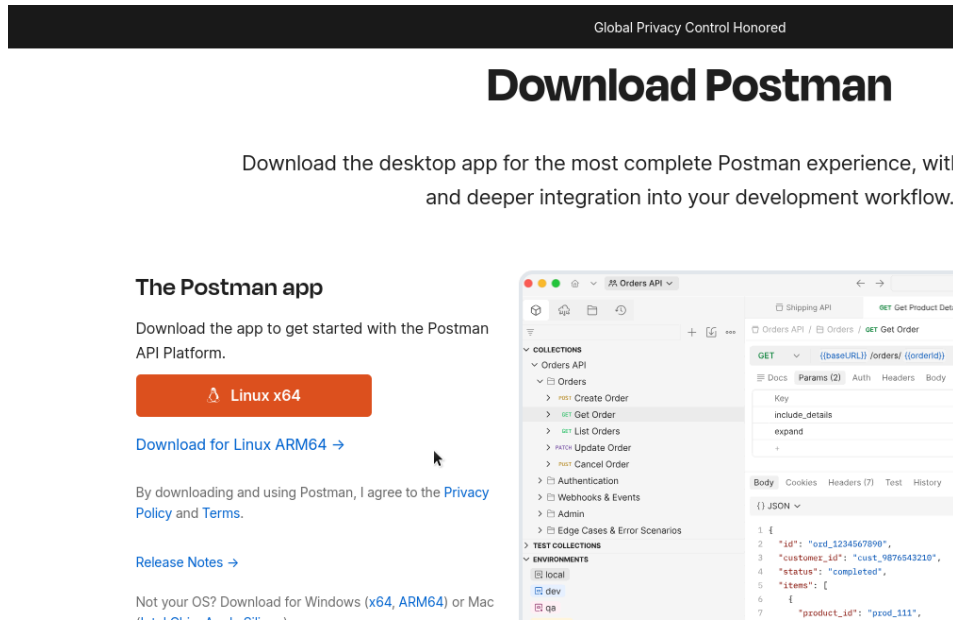
VAmPI Installation via Docker

Deploy VAmPI to simulate a vulnerable REST API environment.

```
> ~
Z) docker run -p 5000:5000 erev0s/vampi:latest
Unable to find image 'erev0s/vampi:latest' locally
latest: Pulling from erev0s/vampi
589002ba0eae: Pull complete
811797f98545: Pull complete
8d1e490ed26d: Pull complete
0b03716e5e03: Pull complete
eacaf042513e: Pull complete
4f4fb700ef54: Pull complete
2bc1971748dc: Pull complete
2ea2995a6afa: Pull complete
Digest: sha256:0a5a224b6e14ae7da6a6ea265178ff71286ff903aec74adee98f660bb0e4ca12
Status: Downloaded newer image for erev0s/vampi:latest
* Serving Flask app 'config'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 138-932-309
```

Installing Postman

Postman is downloaded from their official site.



Then we can extract and run the executable.

```
>>>> ~/Downloads/postman
• [avd] Z> ls
📁 postman-linux-x64.tar.gz
```

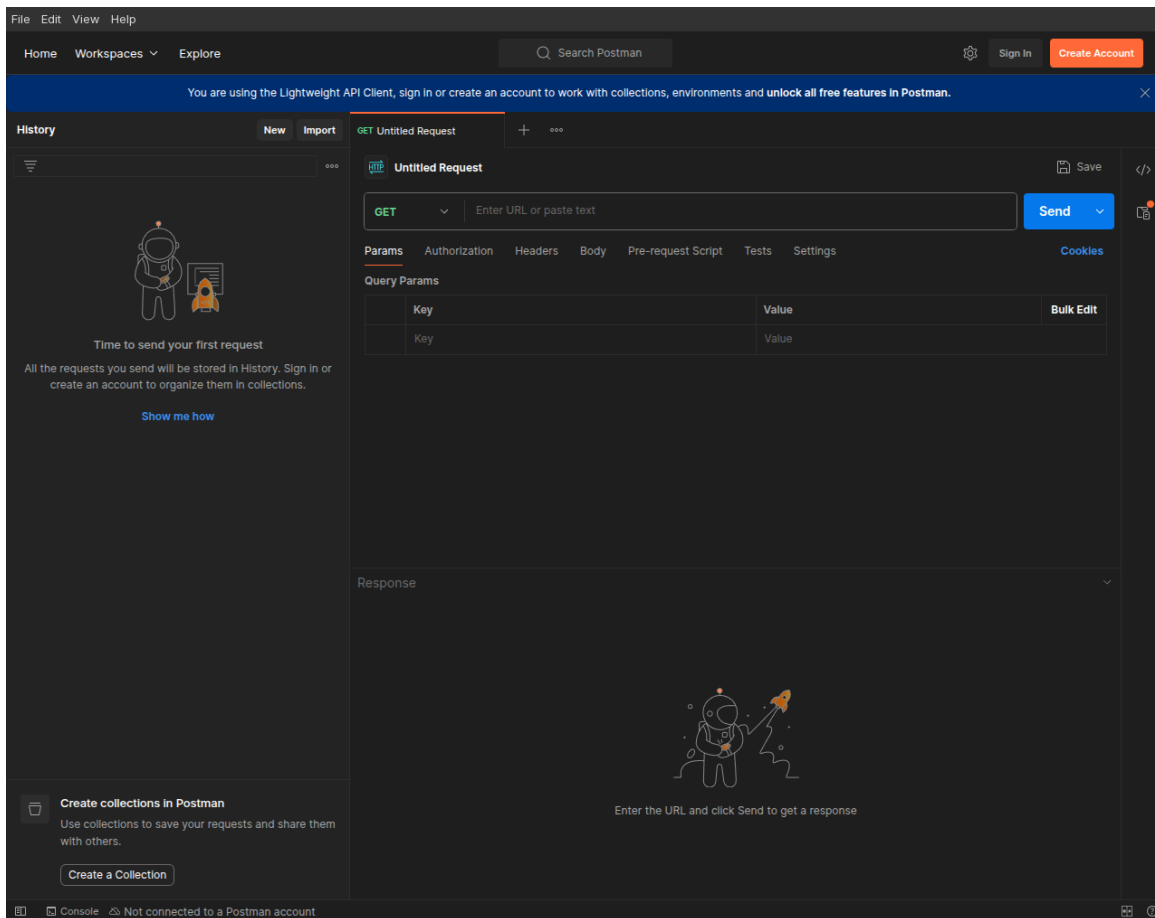
```
>>>> ~/Downloads/postman
• [avd] Z> tar -xf postman*
```

```
>>>> ~/Downloads/postman took 2s
• [avd] Z> ls
📁 Postman 📁 postman-linux-x64.tar.gz
```

```

>>>> Downloads/postman/Postman
• [avd] Z> ./Postman
[436408:0530/114724.240016:ERROR:src/dbus/bus.cc:408] Failed to connect to th
e bus: Failed to connect to socket /run/dbus/system_bus_socket: No such file
or directory
[436408:0530/114724.389907:ERROR:src/dbus/bus.cc:408] Failed to connect to th
e bus: Failed to connect to socket /run/dbus/system_bus_socket: No such file
or directory
[436408:0530/114724.389966:ERROR:src/dbus/bus.cc:408] Failed to connect to th
e bus: Failed to connect to socket /run/dbus/system_bus_socket: No such file
or directory
[436408:0530/114724.389999:ERROR:src/dbus/bus.cc:408] Failed to connect to th
e bus: Failed to connect to socket /run/dbus/system_bus_socket: No such file
or directory
[436408:0530/114724.390029:ERROR:src/dbus/bus.cc:408] Failed to connect to th
e bus: Failed to connect to socket /run/dbus/system_bus_socket: No such file
or directory
[436408:0530/114724.390060:ERROR:src/dbus/bus.cc:408] Failed to connect to th
e bus: Failed to connect to socket /run/dbus/system_bus_socket: No such file
or directory
[PerformanceTestV2] [DuckDBService] Using centralized DB path: /home/loystonp
ais/.config/Postman/duckdb/results.db

```



```
api.json X
home > loystonpals > Downloads > postman > {} api.json > {} Item > {} 0 > {} request > {} url > raw
9
10  "item": [
11    {
12      "event": [
13        {
14          "script": {
15            "type": "text/javascript"
16          }
17        }
18      ],
19      "request": {
20        "method": "GET",
21        "header": [
22          {
23            "key": "Accept",
24            "value": "application/json"
25          }
26        ],
27        "url": {
28          "raw": "{{baseUrl}}/createdb",
29          "host": [
30            "{{baseUrl}}"
31          ],
32          "path": [
33            "createdb"
34          ]
35        }
36      ]
37    }
38  ]
39  }
```

File Edit View Help

Loyston Pals's Workspace

Overview Overview VAmPI + No environment

AI Invite Upgrade

NEW Chat

Start using Agent Model
Your plan includes 50 AI credits per month to use Agent Mode.

VAmPI

Overview Authorization Scripts Tests Variables Runs

0 0

GET Creates and populates the database wit...
GET VAmPI home
GET Retrieves user by username
GET Retrieves all users
GET Retrieves all details for all users (debug)
GET Retrieves all books
POST Register new user
POST Login to VAmPI
GET Retrieves currently logged in user
POST Add new book
GET Retrieves book by title along with secret
PUT Update users email
PUT Update users password
DELETE Deletes user by username (Only Admins)

OpenAPI v3 specs for VAmPI

[View complete documentation](#)

Mitigations

- **Robust Authorization and Validation**

- Implement explicit object-level authorization checks to prevent unauthorized data access (BOLA).

- **Secure Data Handling**

- Use allow-lists for input parameters to prevent Mass Assignment attacks.

- **Inventory and Lifecycle Management**

- Regularly audit and decommission legacy API versions to prevent improper inventory management.

Conclusion

API security is a multifaceted challenge that requires moving beyond traditional web testing methodologies. By understanding API architecture and utilizing tools like Postman and VAmPI, security professionals can effectively identify and mitigate high-impact vulnerabilities like BOLA and Mass Assignment. A proactive approach involving thorough documentation analysis, lab-based practice, and robust authorization frameworks is essential for securing modern interconnected systems.

References

<https://github.com/erev0s/VAmPI>

<https://www.postman.com/>

<https://owasp.org/www-project-api-security/>